```
import axios from "axios";

const BASE_URL = process.env.NEXT_PUBLIC_BACKEND_URL ||
"http://localhost:5002";

const hospitalDoctorApi = axios.create({
    baseURL: BASE_URL,
});

const getDoctorToken = () => {
    if (typeof window === 'undefined') return null;
    return localStorage.getItem('hospitalDoctorToken') ||
           localStorage.getItem('doctorToken') ||
           localStorage.getItem('token');
};

hospitalDoctorApi.interceptors.request.use(
    (config) => {
        const token = getDoctorToken();
        if (token) {
            config.headers.Authorization = `Bearer ${token}`;
        }
        return config;
    },
    (error) => {
        return Promise.reject(error);
    }
);

const HospitalDoctorAPI = {
    // 1. Login
    login: async (credentials) => {
        try {
            const response = await
hospitalDoctorApi.post('/api/hospital/doctors/login', credentials);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Login failed");
        }
```

```
    },

    // 2. Doctor Dashboard Overview
    getDashboard: async () => {
        try {
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/dashboard');
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch dashboard data");
        }
    },

    // 3. Assigned Cases Listing supporting multi-specialist
collaboration tabs
    getCases: async (tabOrType = "active", legacyStatus = "") => {
        try {
            const params = {};
            const standardTabs = ['active', 'pending', 'discharge',
'history', 'bedside', 'pending-bedside'];

            if (standardTabs.includes(tabOrType)) {
                params.tab = tabOrType;
            } else {
                // Graceful legacy fallback mapper
                if (legacyStatus === 'Pending Handovers') params.tab =
'pending';
                else if (legacyStatus === 'In-Progress') params.tab =
'active';
                else if (legacyStatus === 'Completed') params.tab =
'history';
                else params.tab = 'active';
            }

            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/cases', {
                params
            });
            return response.data;
        } catch (error) {
```

```
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch cases");
        }
    },

    // 4. Patient Full Details
    getCaseDetails: async (id) => {
        try {
            const response = await hospitalDoctorApi.get(`/hospital-
doctor/panel/case-details/${id}`);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch case details");
        }
    },

    // 5. Process & Create Prescription (Multipart FormData support
corrected)
    addPrescription: async (formData) => {
        try {
            let url = '/hospital-doctor/panel/prescription/add';

            // Extract appointmentId dynamically to satisfy backend
query parameters fallback
            if (formData && formData instanceof FormData) {
                const appointmentId = formData.get('appointmentId');
                if (appointmentId) {
                    url += `?
appointmentId=${encodeURIComponent(appointmentId)}`;
                }
            }

            const response = await hospitalDoctorApi.post(url,
formData);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to create prescription");
        }
    },
```

```
    // 6. Get Hospital Colleagues
    getColleagues: async () => {
        try {
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/colleagues');
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch colleagues");
        }
    },

    // 7. Transfer / Handover Patient
    transferCase: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/transfer', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to transfer patient");
        }
    },

    // 8. Accept Patient Handover (Doctor B Action)
    acceptTransfer: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/accept-transfer', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to accept patient handover");
        }
    },

    // 9. Reject Patient Handover
    rejectTransfer: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
```

```
doctor/panel/case/reject-transfer', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to reject patient handover");
        }
    },

    // 10. Submit Discharge Summary
    submitDischargeSummary: async (body) => {
        try {
            let data = body;
            let appointmentId = body?.appointmentId;

            // Defensive wrapper: standard objects passed as multipart
content can fail parse validation.
            // If the body is a plain JSON object, convert it to
FormData programmatically.
            if (body && !(body instanceof FormData)) {
                data = new FormData();
                Object.keys(body).forEach(key => {
                    data.append(key, body[key]);
                });
            } else if (body instanceof FormData) {
                appointmentId = body.get('appointmentId');
            }

            const url = appointmentId
                ? `/hospital-doctor/panel/case/discharge-summary?
appointmentId=${encodeURIComponent(appointmentId)}`
                : '/hospital-doctor/panel/case/discharge-summary';

            const response = await hospitalDoctorApi.post(url, data, {
                headers: {
                    'Content-Type': 'multipart/form-data'
                }
            });
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to submit discharge summary");
```

```
        }
    },

    // 11. Duty Status Toggle
    toggleDutyStatus: async (status) => {
        try {
            const response = await hospitalDoctorApi.patch('/hospital-
doctor/panel/status/duty-toggle', { status });
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to update duty status");
        }
    },

    // 12. Get Template Medicines List
    getMedicines: async () => {
        try {
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/medicines');
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch medicines");
        }
    },

    // 13. Update Clinical Summary
    updateClinicalSummary: async (id, body) => {
        try {
            const response = await hospitalDoctorApi.put(`/hospital-
doctor/panel/case/clinical-summary/${id}`, body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to update clinical summary");
        }
    },

    // ========================================
    // Specialist Bedside Care Team Endpoints
```

```
    // ========================================

    // 14. Request Bedside Specialist (Primary Doctor Action)
    requestBedsideHelp: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/bedside-request', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to request bedside specialist");
        }
    },

    // 15. Respond to Bedside Request (Specialist Action:
Accept/Decline)
    respondBedsideRequest: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/bedside-respond', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to respond to bedside request");
        }
    },

    // 16. Submit Specialist Observation Feedback (Specialist Action)
    submitBedsideFeedback: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/bedside-feedback', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to submit bedside observation feedback");
        }
    },

     // ========================================
    // Profile Management Endpoints
```

```
    // ========================================

    // 17. Get Doctor Profile Details (Page Load)
    getProfile: async () => {
        try {
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/profile');
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch profile details");
        }
    },

    // 18. Update Doctor Profile (Multipart data support)
    updateProfile: async (formData) => {
        try {
            const response = await hospitalDoctorApi.put('/hospital-
doctor/panel/profile/update', formData, {
                headers: {
                    'Content-Type': 'multipart/form-data'
                }
            });
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to update profile");
        }
    },

    // 19. Get Specializations list
    getSpecializations: async () => {
        try {
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/specializations');
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch specializations");
        }
    },
```

```
    // 20. Get Admin Qualifications list
    getQualifications: async () => {
        try {
            const response = await hospitalDoctorApi.get('/admin/doctor-
data/qualifications');
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch qualifications");
        }
    },

    // 21. Start Specialist Bedside Shift
    startBedsideShift: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/bedside-start', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to start bedside shift");
        }
    },

    // 22. Finish/Complete Specialist Bedside Shift
    completeBedsideShift: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/bedside-complete', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to complete bedside shift");
        }
    },

    // 23. Get Discharge Summary Print Data from Database
    getDischargePrintData: async (id) => {
        try {
            const response = await hospitalDoctorApi.get(`/hospital-
```

```
doctor/panel/case/discharge-summary/print/${id}`);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch discharge summary print details");
        }
    },
     // 24. Fetch Complete Consultation History List
    getHistoryList: async (page = 1, limit = 10, search = "") => {
        try {
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/cases/history-list', {
                params: { page, limit, search }
            });
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch consultation history list");
        }
    },
     // 3c. Fetch Admission Cases Listing supporting multi-specialist
collaboration tabs
    getAdmissionCases: async (tabOrType = "active", legacyStatus = "")
=> {
        try {
            const params = {};
            const standardTabs = ['active', 'pending', 'discharge',
'history', 'bedside', 'pending-bedside'];

            if (standardTabs.includes(tabOrType)) {
                params.tab = tabOrType;
            } else {
                // Graceful legacy fallback mapper
                if (legacyStatus === 'Pending Handovers') params.tab =
'pending';
                else if (legacyStatus === 'In-Progress') params.tab =
'active';
                else if (legacyStatus === 'Completed') params.tab =
'history';
                else params.tab = 'active';
            }
```

```
            const response = await hospitalDoctorApi.get('/hospital-
doctor/panel/cases/pending-admissions', {
                params
            });
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to fetch admission cases");
        }
    },
      // 26. Self-Assign Case (Doctor Panel)
    selfAssignCase: async (body) => {
        try {
            const response = await hospitalDoctorApi.post('/hospital-
doctor/panel/case/self-assign', body);
            return response.data;
        } catch (error) {
            return Promise.reject(error.response?.data?.message ||
"Failed to self-assign case");
        }
    }
};

export default HospitalDoctorAPI;
```